

CS496 AI / ML / Agentic AI Inline Diagrams

Exam-ready study guide built from the provided slides and code-demo documents

Exam-important: Study this as a concept map, not as a script. The exam-critical flow is: AI -> ML -> Deep Learning -> Generative AI/LLMs -> Agentic AI, then how models/agents are built with LangChain, LangGraph, AutoGen, and CrewAI.

1. Source inspection

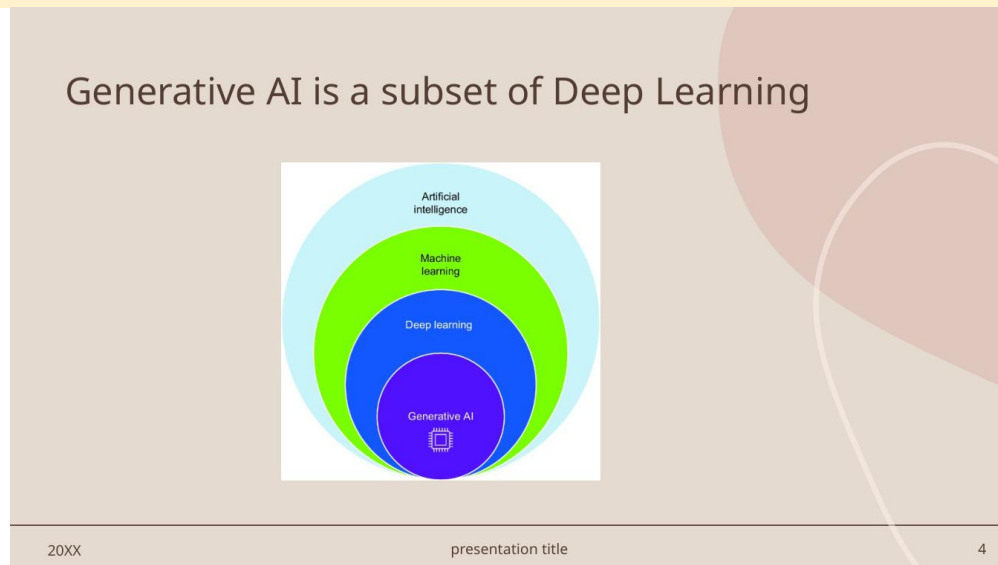
Relevant source	Size	What is covered
010 Generative AI.pptx	27 slides	Generative AI, GANs, VAEs, GPT/ChatGPT, prompts, tuning
011 Getting Started With LangChain.pptx	17 slides	LangChain setup, LLM/chat interfaces, prompt templates, LangGraph
_012_MLP-How it Works -(MultiLayer Perceptrons MLP) Example.pptx	23 slides	MLP structure, perceptron math, activation, weights, vectorization, gradient descent
013 The Context-Machine Learning - Overview.pptx	17 slides	AI/ML/DL context, rule learning, biology analogy, loss and training
014 The Agentic AI Revolution.pptx	20 slides	Agents, autonomy, history, architecture, cognitive loop
015 Demo -Cognitive Loop-LangChain Comprehensive Example-2 .docx	31 pages	Full cognitive loop code implementation
016 LngChain Multiagent.docx	8 pages	LangChain multi-agent roles and orchestrator
017 Autogen Single Agent Formatted.docx	10 pages	AutoGen single assistant/user proxy examples
018 AutoGen MultiAgent.docx	10 pages	AutoGen group/sequential/coding multi-agent examples
019 Crew Single Agent Formatted.docx	14 pages	CrewAI single content agent examples
020 CrewAI MultiAgent.docx	20 pages	CrewAI multi-agent marketing team examples

2. _010 Generative AI - The precursor of Agentic AI

This deck introduces generative AI as the content-creation layer that later enables agentic systems. The order is: definition -> deep learning foundation -> major generative model families -> GPT/ChatGPT -> pretraining, prompts, and tuning.

- Generative AI means models that generate new content such as text, images, code, or datasets.
- The model is not explicitly programmed with each output. It learns patterns from training data and uses those patterns to create new examples.
- The main goal of a generative model is to learn the data distribution well enough to generate similar but new data points.
- The deck connects generative AI to deep learning and word embeddings because embeddings let language become numerical vectors that neural networks can process.

Exam-important: Generative AI = learned pattern generation. It is not the same as ordinary classification, which assigns labels to existing inputs.



Source diagram - Slide 4: Generative AI as a subset of deep learning

How to read it: Read the nested circles from outside to inside: AI is the broad field, machine learning is inside AI, deep learning is inside ML, and generative AI is a deep-learning-centered branch. The exact names may vary, but the exam idea is the hierarchy.

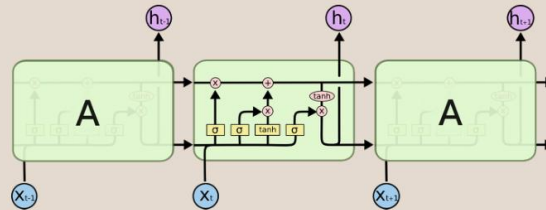
2.1 Large Language Models and sequence models

An LLM is a large model trained on huge amounts of text/data. The Big Data era, GPUs/cloud hardware, and scalable neural architectures made these models practical. LLMs generate language by predicting token sequences.

Model family	Core idea	When it is used / exam meaning
RNN / LSTM	Processes sequences step by step and keeps state across time.	Earlier sequence generation: sentences, paragraphs, time series. LSTM improves long-term dependency handling over basic RNNs.
GAN	Generator competes against discriminator.	Image/content generation through adversarial training.
VAE	Encoder learns probabilistic latent space; decoder samples/generates.	Generates data by sampling latent representations.
Transformer / GPT	Uses self-attention to model token relationships across context.	Modern LLM text generation, summarization, translation, QA, coding help.

Sequential LSTM Models

- **Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM):** These are types of neural networks that can be used for sequence generation tasks, such as generating sentences, paragraphs, or even entire articles.



20XX

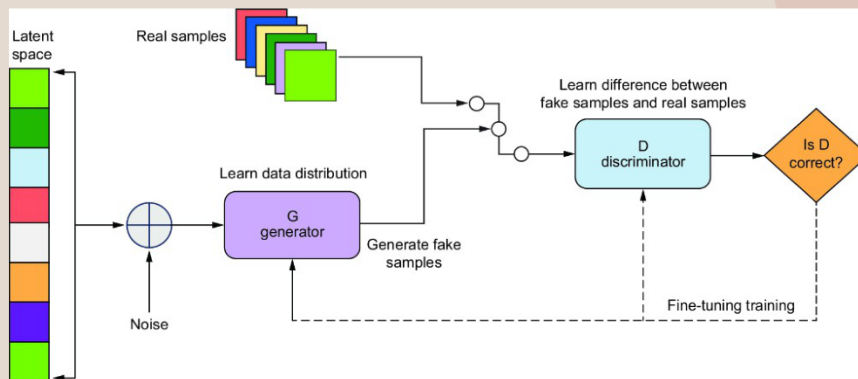
presentation title

7

Source diagram - Slide 7: Sequential LSTM model

How to read it: The same recurrent block is reused across time steps. Each step reads current input and previous hidden state, so earlier context can influence later output. LSTM adds gates to manage what is remembered or forgotten.

GAN



20XX

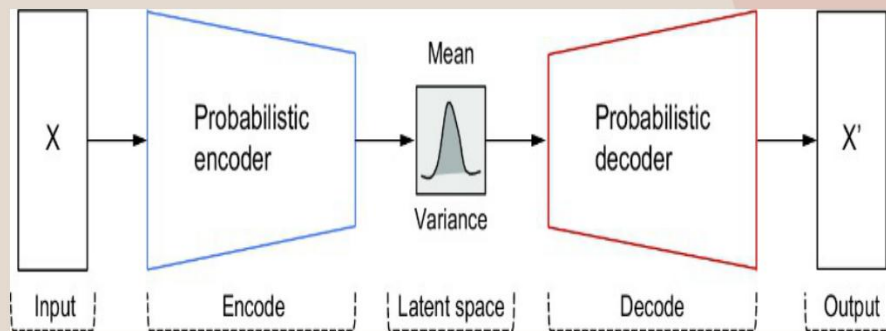
presentation title

9

Source diagram - Slide 9: GAN architecture

How to read it: Noise enters the generator, which produces fake samples. Real and fake samples go to the discriminator. The discriminator learns real-vs-fake; the generator learns to fool it. Training improves both networks iteratively.

VAE's



20XX

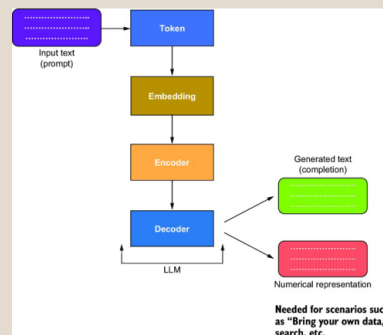
presentation title

11

Source diagram - Slide 11: VAE architecture

How to read it: Input is encoded into a probabilistic latent space, usually represented by a mean and variance. A sample from that latent distribution is decoded back into output. Generation comes from sampling the learned latent space.

The Transformer



20XX

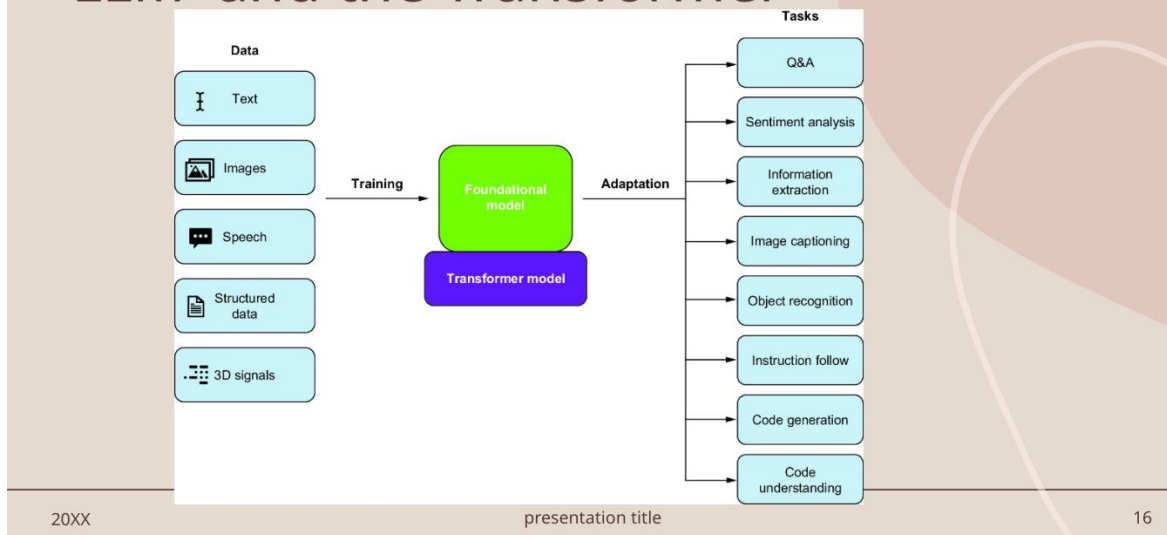
presentation title

15

Source diagram - Slide 15: Transformer structure

How to read it: Transformer processing is built from embeddings, attention, feed-forward layers, and output prediction. The most important exam idea is self-attention: each token can use information from other tokens in the sequence.

LLM and the Transformer



Source diagram - Slide 16: LLM and transformer pipeline

How to read it: Inputs such as text, images, numbers, or code are represented as vectors, processed by the transformer, and turned into outputs such as labels, generated text, summaries, or answers. The diagram shows why transformers became general-purpose AI building blocks.

2.2 GPT, ChatGPT, and transformer training

- GPT stands for Generative Pre-trained Transformer.
- Generative: produces text. Pre-trained: first trained on massive general data. Transformer: uses the transformer architecture, especially self-attention.
- GPT-style models are autoregressive: they predict the next token from previous tokens.
- Masked language modeling, used by BERT-style models, hides tokens and asks the model to predict the missing tokens. GPT-style training instead predicts the next token.
- Tokenization converts text into tokens such as words, subwords, punctuation, or symbols before the model processes it.

Exam-important: Do not confuse GPT training with BERT training. GPT = next-token/autoregressive prediction. BERT = masked token prediction.

Correction / slide note: The slide text describes ChatGPT as an instance of GPT-4. Safer exam wording: ChatGPT is a conversational application built on GPT-family transformer models. The backing model can vary by product/version.

LLM generation flow

Text corpus
 -> tokenization
 -> embeddings
 -> transformer blocks with self-attention
 -> probability distribution over next token
 -> generated sequence

2.3 Prompts, prompt engineering, and tuning

Concept	Meaning	Exam distinction
Prompt	The input text/instruction sent to the model.	Changes the model input, not model weights.
Prompt engineering	Designing instructions/examples to get better outputs.	Useful when fine-tuning is not feasible.
Few-shot learning	Putting examples in the prompt.	Teaches format/behavior in context.
Pretraining	Initial large-scale training on broad data.	Learns general language patterns.
Fine-tuning	Additional training on a task/domain dataset.	Changes weights to specialize behavior.
RLHF	Human feedback ranks/scores outputs; model is optimized	Aligns behavior with human preferences.

	toward preferred responses.	
Domain tuning	Fine-tuning on legal, medical, technical, or other domain data.	Improves specialized accuracy if dataset quality is good.
Hyperparameter tuning	Adjusting learning rate, batch size, layer choices, etc.	Affects training efficiency/stability.

Exam-important: Fine-tuning changes the model. Prompt engineering changes the input. RLHF changes behavior using human preference feedback.

Prompt categories from slides

Direct question: What is the capital of Finland?
Instruction-based: Write a summary of 1984 by George Orwell.
Creative generation: Compose a poem about the ocean.
Problem solving: How can I fix a leaking faucet?
Coding help: Write a Python function to calculate factorial.

3. _011 Getting Started With LangChain

This deck shows how LangChain wraps LLM providers, prompts, chat messages, runnable chains, and LangGraph workflows. Treat LangChain as the application-building framework around models, not as the model itself.

Exam-important: LangChain is not an LLM. It is a framework for building LLM applications by connecting model calls, prompts, tools, memory, and workflows.

Installation shown in the slides

```
pip install langchain langchain-openai langchain-community
pip install langchain-text-splitters langchain-postgres
```

Interface	Input style	Use case
LLM interface	Usually one string prompt.	Simple completion-style calls.
Chat model interface	List of role-based messages.	Chat assistants with system/user/assistant roles.

Slide LLM example

```
from langchain_openai.llms import OpenAI
model = OpenAI(model="gpt-3.5-turbo")
model.invoke("The sky is")
```

Correction / slide note: Current LangChain APIs commonly use ChatOpenAI for chat models such as GPT-3.5/4-style chat models. The slide uses OpenAI(model="gpt-3.5-turbo"), which may be version-dependent or problematic depending on library version. Exam idea: use the correct model wrapper for completion vs chat.

Chat model example

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.messages import HumanMessage

model = ChatOpenAI()
prompt = [HumanMessage("What is the capital of France?")]
model.invoke(prompt)
```

Message type	Role	Meaning
HumanMessage	user	Message from the human/user.
AIMessage	assistant	Message from the assistant/model.
SystemMessage	system	Higher-priority instruction describing behavior/rules.
ChatMessage	custom	Arbitrary role message when needed.

Exam-important: System messages define behavior; human messages ask the current question. In structured chat prompts, roles matter.

3.1 Prompt templates and chatbot pipeline

Prompt/chaining idea

```
PromptTemplate: variables such as {context} and {question}
ChatPromptTemplate: role-based messages with variables
RunnableLambda: wraps your own function in the LangChain Runnable interface
```

Typical chain:
input values -> prompt template -> model -> response

Exam-worthy prompt pattern

Answer the question based on the context below.
If the answer is not in the context, answer "I don't know".

```
Context: {context}
Question: {question}
Answer:
```

3.2 LangGraph

- LangGraph is built on top of LangChain.
- LangChain chains are often DAG-like: data moves forward without loops.
- LangGraph adds stateful graphs and cycles, which are useful for agent behavior.
- Nodes are functions/computation steps. Edges connect nodes. Conditional edges choose the next node dynamically. START and END define entry/exit.

Exam-important: LangGraph matters for agents because agents often need loops: think -> choose tool -> observe -> think again -> stop when done.

A Very Simple Example

```
from langgraph.graph import StateGraph, START, END
# Define agents
def research_node(state):
    # Research logic here
    return {"messages": [{"content": "Research complete", "name": "researcher"}]}

def chart_node(state):
    # Chart generation logic here
    return {"messages": [{"content": "Chart generated", "name": "chart_generator"}]}

# Build multi-agent graph
workflow = StateGraph()
workflow.add_node("researcher", research_node)
workflow.add_node("chart_generator", chart_node)
workflow.add_conditional_edges("researcher", lambda state: "chart_generator",
                               {"chart_generator": "chart_generator"})
workflow.add_edge(START, "researcher")
workflow.add_edge("chart_generator", END)
```

16

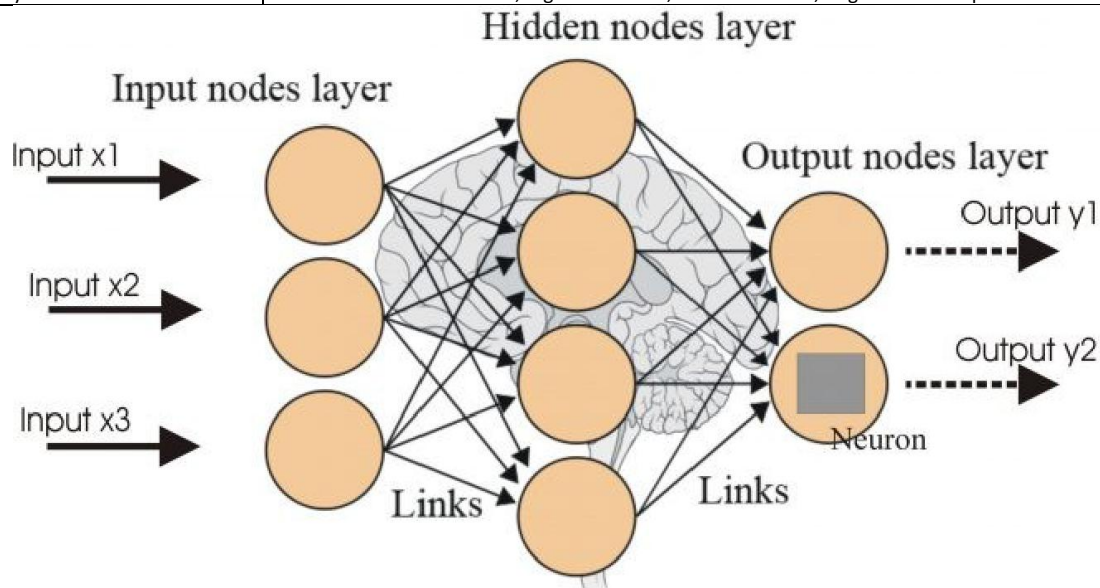
Source code slide - Slide 16: simple LangGraph workflow

How to read it: START goes to a researcher node. A conditional edge routes from researcher to chart_generator. The chart node then goes to END. This is the simplest version of a stateful graph workflow.

4. _012 MLP - How it Works

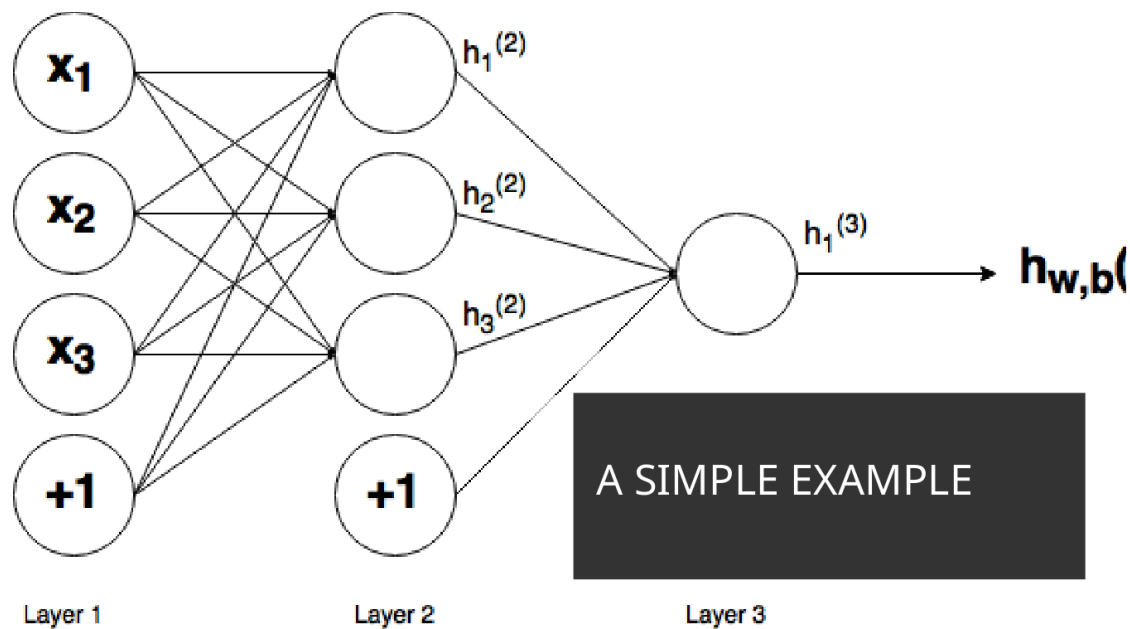
This deck explains the mechanics of a basic feedforward neural network using a multilayer perceptron (MLP). The core ideas are layers, weights, bias, activation, loss, backpropagation, vectorization, and gradient descent.

Layer type	Role
Input layer	Receives raw features. Each input neuron corresponds to a feature.
Hidden layer(s)	Perform weighted transformations and nonlinear activations. More hidden layers allow more complex representations.
Output layer	Produces final classification, regression value, detection result, or generated output.



Source diagram - Slide 4: input, hidden, output layers

How to read it: Read left to right. Inputs connect to hidden neurons through weighted links, hidden neurons connect to output neurons, and every link has a learnable weight. This is the basic MLP structure.



Source diagram - Slide 7: simple MLP example

How to read it: The $+1$ nodes represent bias inputs. The network maps features x_1 , x_2 , x_3 through a hidden layer to an output h . Bias lets the neuron shift its decision threshold.

Exam-important: Forward pass computes outputs. Loss compares output with target. Backpropagation computes weight/bias responsibility for error. Gradient descent updates parameters.

4.1 Perceptron formula, activation, and learning

Core MLP computation

$$h = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + b$$

Vector form:

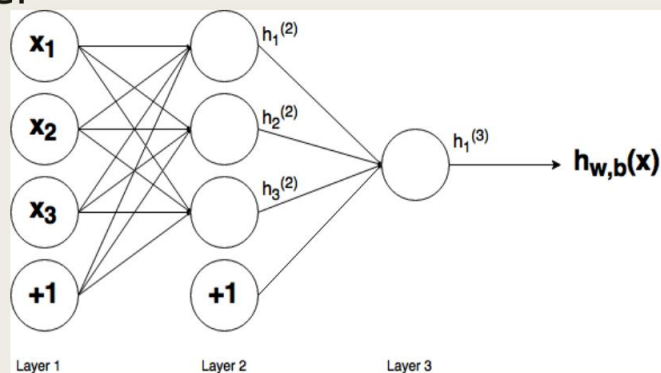
$$z = W * x + b$$

$$h = f(z)$$

- Weights control how strongly each input affects the neuron.
- Bias shifts the activation threshold. It lets the model fit data better even when inputs are zero or centered.
- Activation functions introduce nonlinearity. Without nonlinear activations, stacked layers collapse into one linear transformation.

Correction / slide note: Slide 8 says an activation function controls the learning rate. That is not accurate. The learning rate is a hyperparameter of the optimizer. Activation functions control neuron output shape/nonlinearity.

A Simple MLP with one Hidden Layer



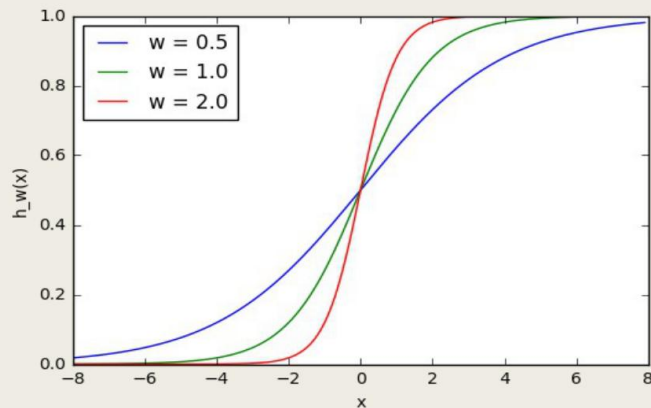
Source diagram - Slide 10: MLP with one hidden layer

How to read it: Each hidden neuron receives all inputs and a bias, computes a weighted sum, applies activation, then feeds the output neuron. This is a fully connected feedforward network.

Sigmoid-style activation shown in the deck

$$f = 1 / (1 + \text{np.exp}(-x * w))$$

Activation function plot



Source graph - Slide 13: activation function plot

How to read it: The sigmoid maps large negative inputs toward 0 and large positive inputs toward 1. Larger weights make the curve steeper, meaning the neuron changes output more sharply around zero.

4.2 Weights, matrices, and vectorization

NumPy representation from slides

```
import numpy as np
w1 = np.array([
    [0.2, 0.2, 0.2],
    [0.4, 0.4, 0.4],
    [0.6, 0.6, 0.6]
])

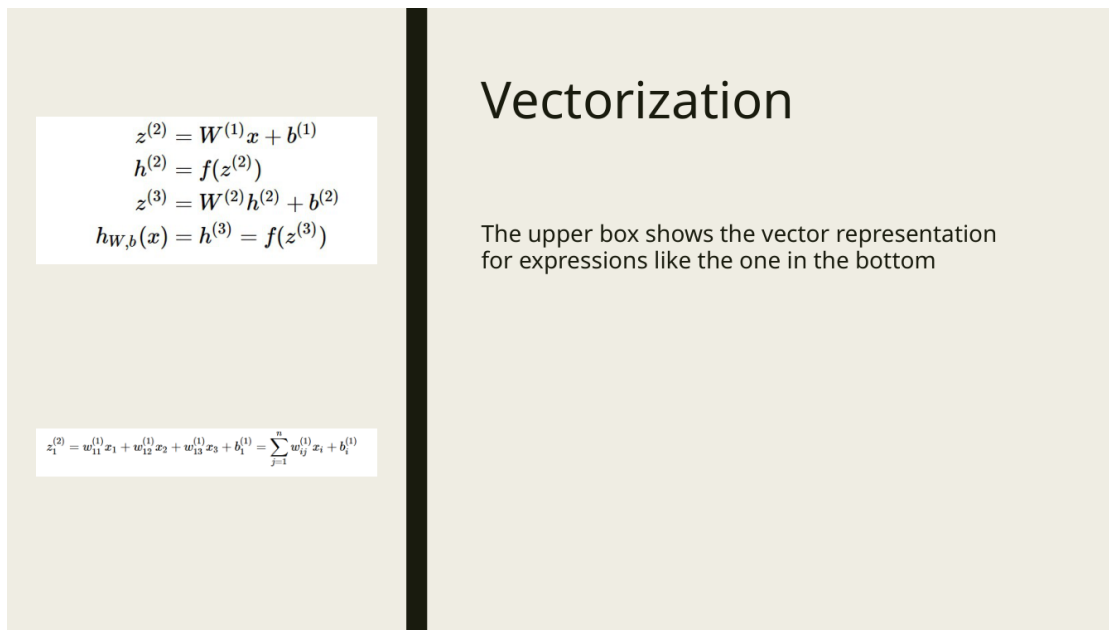
w2 = np.zeros((1, 3))
w2[0, :] = np.array([0.5, 0.5, 0.5])

b1 = np.array([0.8, 0.8, 0.8])
b2 = np.array([0.2])
```

Computing the Weights in MLP's

$$\begin{aligned}h_1^{(2)} &= f(w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3 + b_1^{(1)}) \\h_2^{(2)} &= f(w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{23}^{(1)}x_3 + b_2^{(1)}) \\h_3^{(2)} &= f(w_{31}^{(1)}x_1 + w_{32}^{(1)}x_2 + w_{33}^{(1)}x_3 + b_3^{(1)}) \\h_{W,b}(x) = h_1^{(3)} &= f(w_{11}^{(2)}h_1^{(2)} + w_{12}^{(2)}h_2^{(2)} + w_{13}^{(2)}h_3^{(2)} + b_1^{(2)})\end{aligned}$$

How to read it: The first hidden-layer activations are computed from input features, weights, and biases. The output layer then combines hidden activations with second-layer weights and bias.



Source diagram - Slide 20: vectorization

How to read it: Instead of writing one equation per neuron, pack values into vectors/matrices. This is why deep learning is mostly fast matrix multiplication plus nonlinear functions.

Forward pass implementation from slide 21

```
def matrix_feed_forward_calc(n_layers, x, w, b):
    for l in range(n_layers-1):
        if l == 0:
            node_in = x
        else:
            node_in = h
        z = w[l].dot(node_in) + b[l]
        h = f(z)
    return h
```

4.3 Gradient descent

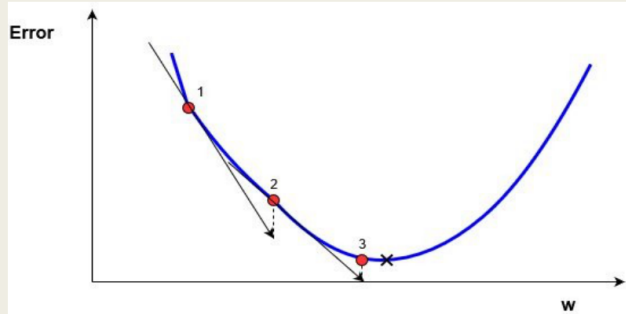
- Gradient descent is an optimization method that repeatedly moves parameters in the opposite direction of the gradient/slope to reduce loss.
- gamma is the learning rate/step size. Too large can overshoot; too small can be slow.

Gradient descent update

```
x_new = x_old - gamma * df(x_old)

# slide example derivative:
def df(x):
    return 4*x**3 - 9*x**2
```

Gradient Descent

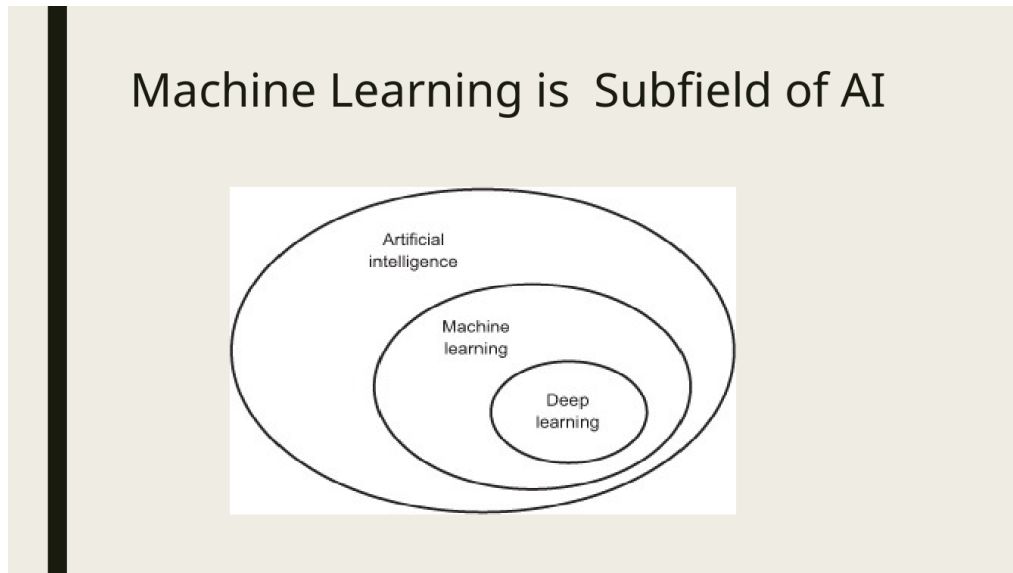


Source diagram - Slide 22: gradient descent

How to read it: The algorithm moves downhill along the curve toward a local minimum. Each update uses the slope at the current point to decide the next step direction.

5. _013 The Context - Machine Learning Overview

This deck places deep learning inside the broader AI and ML landscape and explains why modern AI revived older ideas: more data, more compute, better software, and improved training algorithms.



Source diagram - Slide 3: ML is a subfield of AI

How to read it: Read the circles as containment: AI is the broad field; ML is a subfield of AI; deep learning is a subfield of ML.

It is not that new – Revival Due to Advances in Technology	
Techniques	Year
Neural networks	1943
Backpropagation	1962?
Convolutional Neural Networks	1979
Recurrent Neural Networks	1980
Long-Short-Term-Memory	1997

Source table - Slide 4: AI is not new

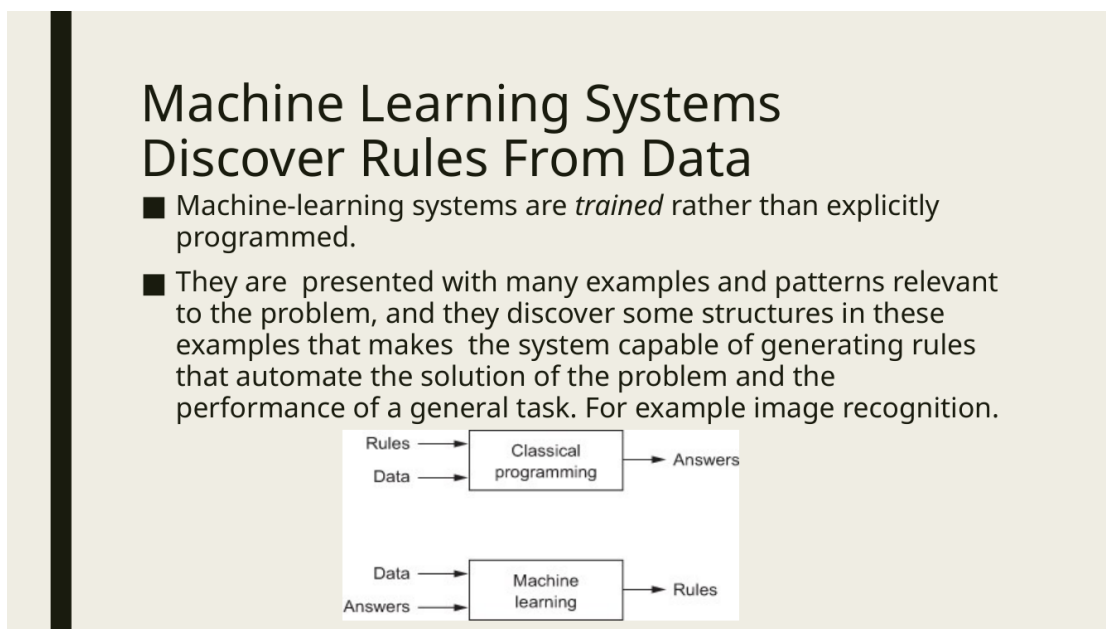
How to read it: The point is not that all techniques are new. Neural networks, backpropagation, CNNs, RNNs, and LSTMs existed earlier. The recent boom comes from hardware, data, and engineering advances.

Field	Definition	Example
AI	Broad effort to automate intellectual tasks normally performed by humans.	Symbolic AI, expert systems, chess systems, planning systems.
Machine Learning	Systems learn useful representations/rules from data using feedback instead of explicit programming.	Image recognition, speech recognition, fraud detection.
Deep Learning	ML using many-layer neural networks to learn successive representations.	CNN image classifiers, transformers, LSTMs, MLPs.

Exam-important: All deep learning is machine learning, but not all machine learning is deep learning. All machine learning is AI, but not all AI is

5.1 Symbolic AI vs machine learning

- Traditional symbolic AI depends on humans writing rules and knowledge bases.
- Machine learning systems are trained from examples, so the system discovers useful patterns from data.
- The slide connects ML to the old Ada Lovelace/Turing question: can machines go beyond only doing exactly what we explicitly ordered?



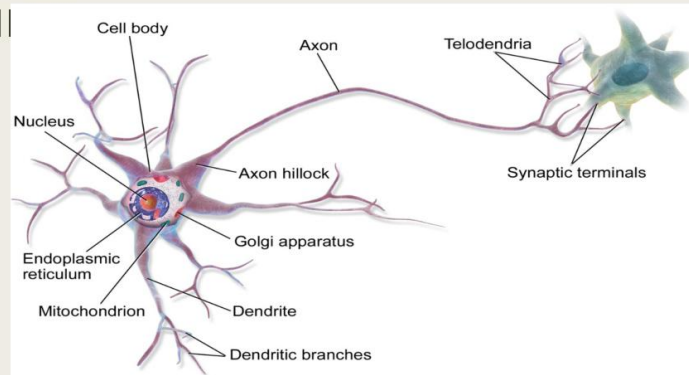
Source diagram - Slide 7: programming vs machine learning

How to read it: Classical programming: rules + data -> answers. Machine learning: data + answers -> learned rules. This is one of the cleanest exam diagrams in the deck.

5.2 Deep learning representations

- Deep learning learns representations across layers. Early layers learn simple patterns; later layers learn more abstract patterns.
- In image models, early layers may learn edges, middle layers parts/shapes, later layers objects/classes.
- Deep learning reduces manual feature engineering because features are learned by the model.

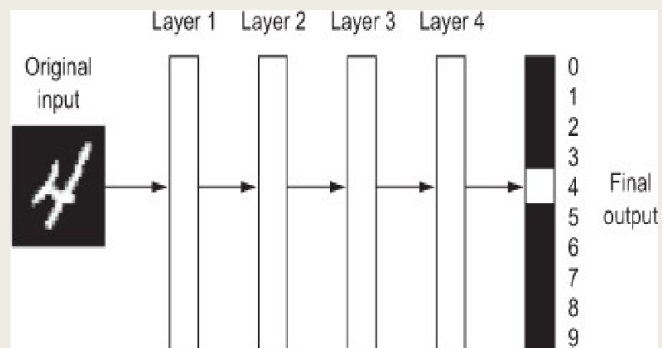
Neural Networks: Lessons From Biology



Source diagram - Slide 10: biological neuron analogy

How to read it: Artificial neurons are inspired by biological neurons, but simplified: inputs arrive, weighted connections transform signals, and outputs pass to later neurons.

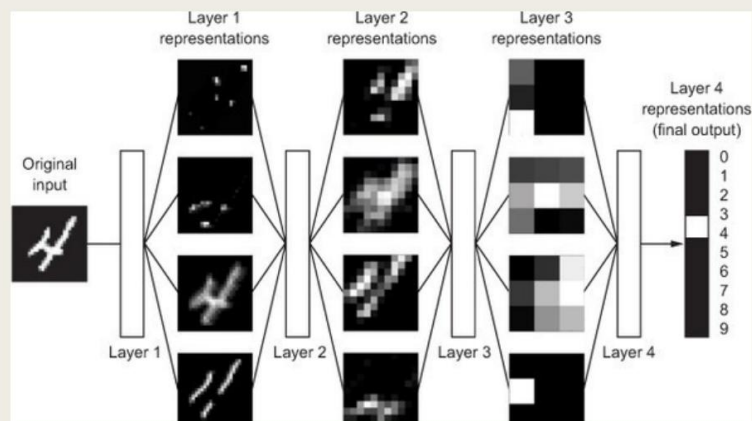
Examples



Source diagram - Slide 12: digit recognition through layers

How to read it: Raw pixels enter layer 1 and are transformed through layers until the final output gives probabilities/classes for digits 0-9. The key idea is layered representation learning.

Another Example

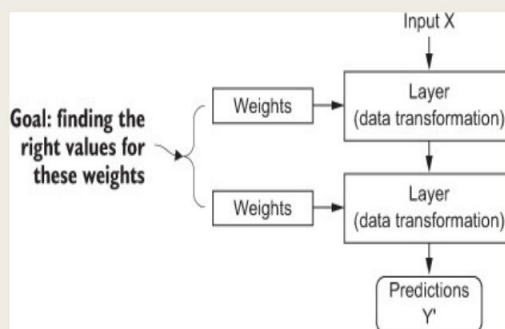


Source diagram - Slide 13: representation hierarchy example

How to read it: Each layer produces more abstract representations of the same digit. This is why deep networks can learn features instead of relying only on manual feature extraction.

5.3 Weights, loss, optimizer, metrics, deployment

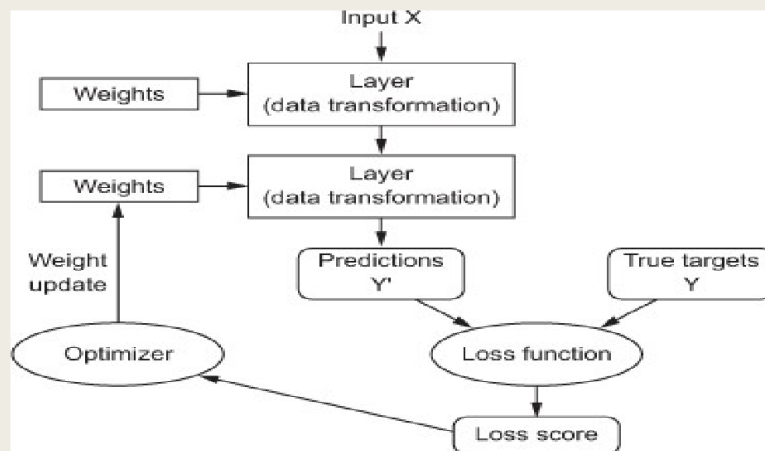
The Weights



Source diagram - Slide 14: weights

How to read it: The network goal is to find useful weight values. Weights define how input data is transformed into predictions.

The Loss Function: The Learning Process



Source diagram - Slide 15: loss function learning process

How to read it: Predictions are compared with true targets through the loss function. The optimizer uses the loss to update weights. This loop repeats during training.

Training component	Meaning
Loss function	Measures how wrong predictions are compared with true targets.
Optimizer	Updates model parameters based on loss/gradients.
Metric	Human-readable performance measure such as accuracy.
Train/test split	Train data teaches the model; test data checks generalization.
Deployment	Saving and using the trained model for real tasks.

6. _014 The Agentic AI Revolution

This deck shifts from models that answer one input to agents that pursue goals across time. The exam focus is what makes a system an agent: perception, state/memory, reasoning, planning, action, learning, autonomy, and tool use.

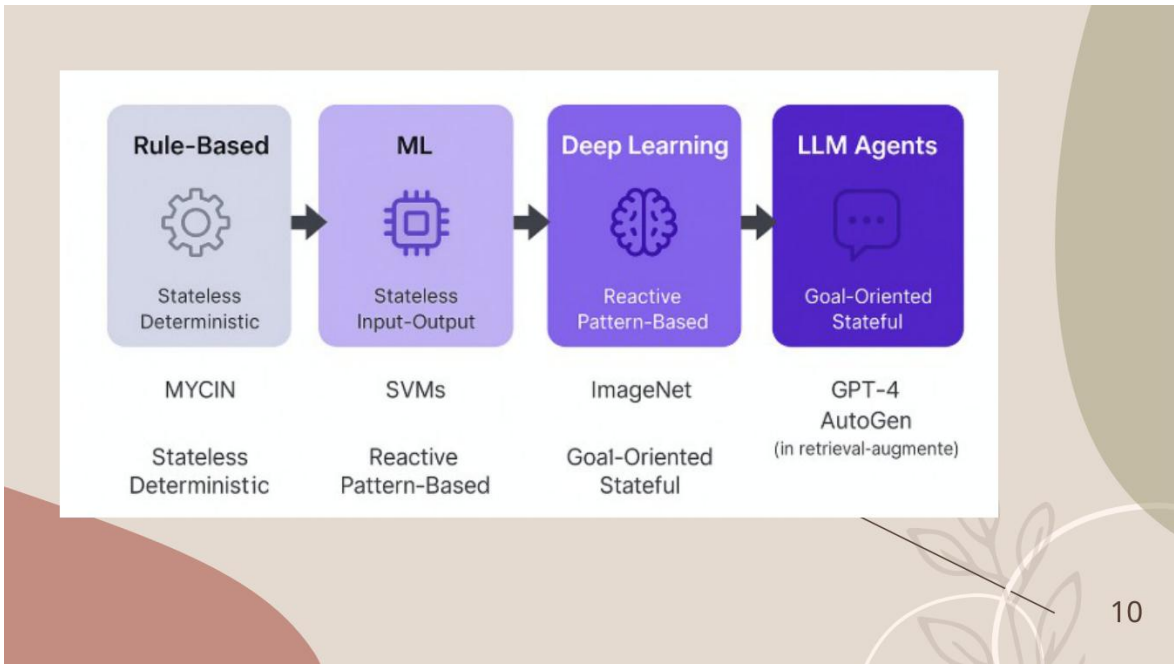
Traditional software	Agentic AI system
Executes predefined instructions.	Pursues goals and chooses actions.
Usually stateless or limited state.	Maintains memory/state across interactions.
Reactive input-output behavior.	Reactive and proactive behavior.
Fixed workflows.	Can plan, adapt, and use tools.
Predictable deterministic pathways.	Makes decisions under uncertainty.

6.1 Traits and definition of agents

Trait	Meaning
Autonomy	Can operate without continuous human guidance.
Persistence	Maintains state/memory over time.
Reactivity	Responds to environment changes.
Proactiveness	Initiates actions based on goals.
Adaptability	Learns from experience and modifies behavior.
Goal-orientation	Plans and reasons to reach objectives.

Exam-important: An agent is not just a chatbot. A real agent senses/perceives, keeps state, reasons, plans, acts through tools/APIs, and learns from outcomes.

6.2 Agent history and evolution



Source diagram - Slide 10: evolutionary trajectory

How to read it: The diagram moves from rule-based systems to classical ML, deep learning, LLM agents, and AutoGen-style agent systems. The right side adds goal-oriented behavior and autonomous agent capabilities.

Era	Main capability	Limitation
1970s-1980s rule systems	Expert systems like MYCIN used logic rules.	Brittle and narrowly scoped.
1990s classical ML	Decision trees/SVMs improved pattern recognition.	Task-specific and mostly stateless.
2010s deep learning	Strong perception: speech, images, translation.	Mostly reactive input-output models.
2020s+ LLMs and agents	Language reasoning, few-shot learning, tool use, planning.	Need memory, verification, orchestration, and safety controls.

6.3 Agent architecture and cognitive loop

- Agent architecture separates concerns: perception, planning, memory, execution/action, and learning.

- LangGraph and CrewAI are examples of frameworks that support these ideas using graphs, workflows, tasks, roles, and state.



Source diagram - Slide 14: cognitive loop

How to read it: The loop cycles through perceive/interpret, reason/decide, act, and learn/update around a goal/context/task objective. The central goal gives direction; memory and feedback make the process adaptive.

Cognitive loop

Perceive -> Reason -> Plan -> Act -> Learn

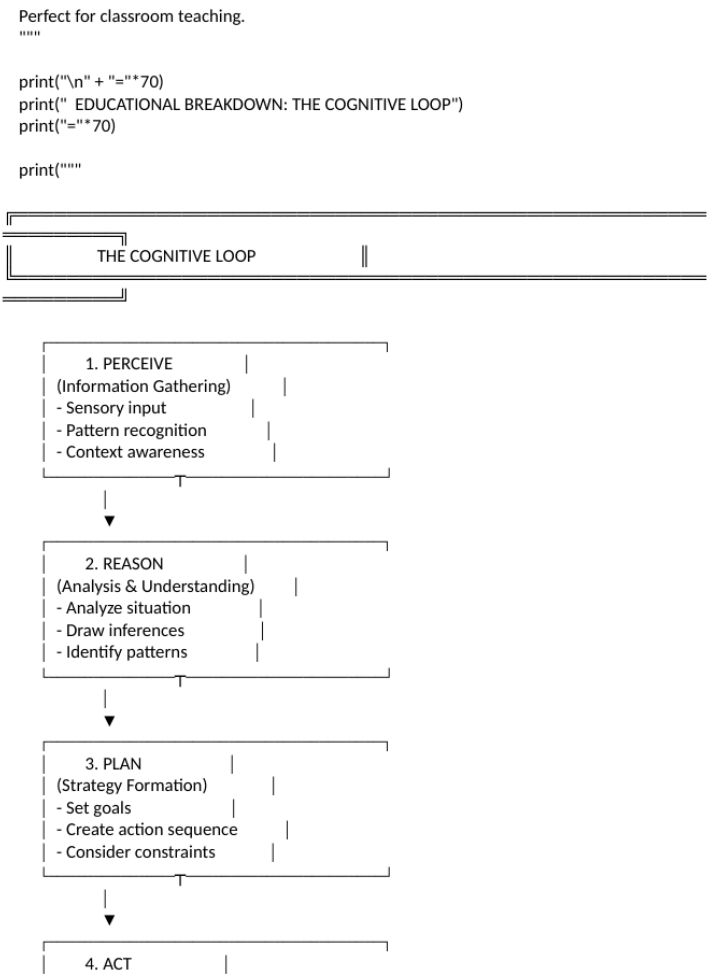
Repeat until the goal is reached, the task fails, or a stopping rule is met.

Phase	What it does	Slide example
Perception	Captures user input, API output, sensors, or external system data and structures it.	message, timestamp, user_id, session_state, sentiment
Reasoning	Interprets the perceived data: intent, priority, meaning, why it matters.	classify_intent; determine_priority using sentiment and history
Planning	Turns reasoning into a sequence of tasks/actions with priorities.	billing_issue -> fetch_account_details -> analyze_billing_history -> generate_explanation -> offer_resolution
Action	Executes through tools, APIs, databases, function calls, or other agents.	billing_api.get_account; llm.generate_response
Learning	Measures outcome and updates memory/preferences/models.	success_score; update_user_preferences; flag low success

Correction / slide note: Slide 17 code appears truncated: it returns {"context": percep}, but the variable is incomplete. The intended idea is to return reasoning context based on perception data.

7. _015 Cognitive Loop - LangChain Comprehensive Example

This 31-page DOCX is the practical code version of the agentic AI cognitive loop. You do not need to memorize every line. You should know the modules, their responsibilities, and the flow through process_input().



Source page screenshot - DOCX page 28: educational cognitive-loop breakdown

How to read it: This page visually connects the same five phases used in the slides: perception, reasoning, planning, action, and learning. It is the implementation map for the CognitiveAgent class.

Code class/module	Responsibility	Key methods / data
PerceptionModule	Processes incoming text and tool outputs; builds situational awareness.	perceive_text_input, perceive_tool_output, get_current_context; detects word count, question mark, sentiment, simple entities.
ReasoningModule	Uses the LLM to analyze needs, missing information, constraints, patterns, and strategy.	analyze_situation, reason_about_approach, _assess_confidence.
PlanningModule	Creates and adapts action plans from reasoning results.	create_plan, adapt_plan, _parse_steps, _estimate_difficulty.
ActionModule	Executes plan steps using registered tools.	execute_plan, _select_tool_for_step; tool matching by keywords.
LearningModule	Stores episodes, extracts knowledge, updates procedural/tool memory, performs meta-learning.	learn_from_interaction, retrieve_relevant_knowledge, get_episodic_context.
MemorySystem	Combines short-term, summary, semantic, episodic, and procedural memory.	ConversationBufferMemory, ConversationSummaryMemory, Chroma semantic memory, episodic list, procedural dict.
CognitiveAgent	Integrates all modules into a full loop.	process_input, _create_tools, _generate_response, reflect_on_experience.

7.1 PerceptionModule

- Maintains a sensory buffer and perception history.
- For text input it stores type, content, timestamp, word count, question detection, sentiment, and simple entity extraction.
- For tool output it stores tool name, output content, timestamp, and success flag based on whether output contains error.

Perception record structure

```
perception = {  
    "type": "text_input",  
    "content": text,  
    "timestamp": datetime.now().isoformat(),  
    "word_count": len(text.split()),  
    "contains_question": "?" in text,  
    "sentiment": self._analyze_sentiment(text),  
    "entities": self._extract_entities(text)  
}
```

7.2 ReasoningModule and PlanningModule

- Reasoning constructs an LLM prompt asking what the user needs, what information exists/missing, constraints, and memory patterns.
- It assesses confidence by counting uncertainty markers such as might/maybe/unclear and confidence markers such as clearly/definitely.
- Planning asks the LLM for specific actions, order of operations, expected outcomes, fallback options, and success criteria.
- If execution fails, adapt_plan prompts the model to revise the plan using execution feedback.

Exam-important: Reasoning answers: what does this mean? Planning answers: what should we do next, in what order?

7.3 ActionModule and tool selection

- ActionModule receives a plan, selects tools for steps, executes them, and records success/details.
- Tools in the example include WebSearch, Calculator, and MemorySearch.
- Tool selection is simplified keyword matching. Production agents should use more reliable routing and validation.

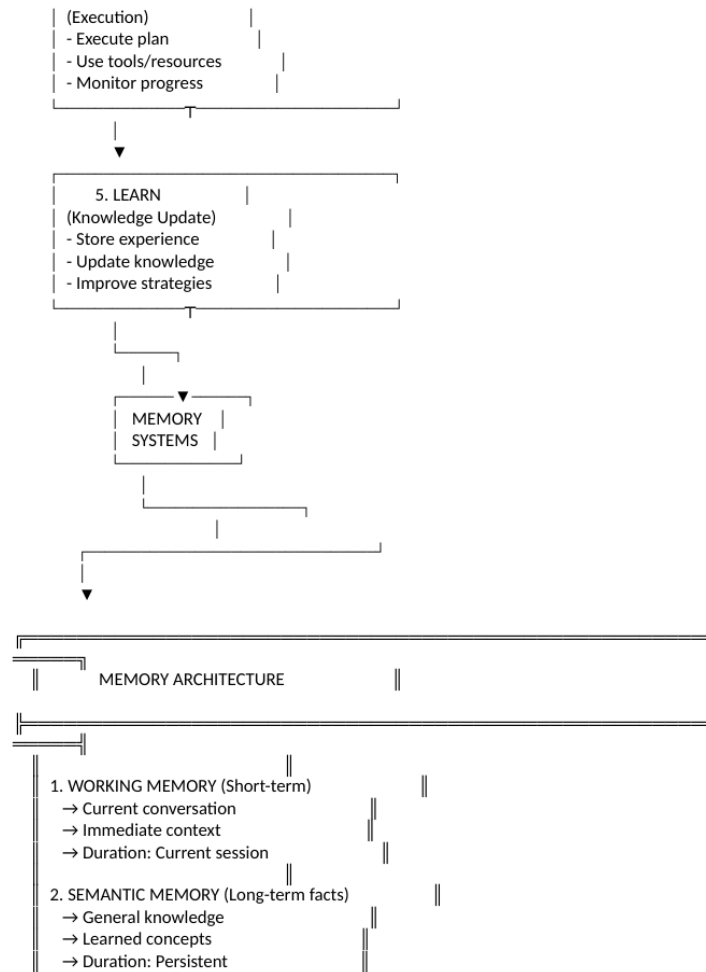
Tool set in the DOCX

Available tools in CognitiveAgent._create_tools():

- WebSearch: simulated search response
- Calculator: evaluates math expressions safely enough for demo, with try/except
- MemorySearch: retrieves relevant semantic memory

7.4 LearningModule and MemorySystem

Memory type	Purpose in agent
Working memory	Immediate conversation context.
Summary memory	Compressed recent history to preserve context without unlimited tokens.
Semantic memory	Long-term facts/knowledge stored and retrieved by embeddings/vector search.
Episodic memory	Specific experiences/interactions and outcomes.
Procedural memory	Strategies, tool effectiveness, and learned ways of doing tasks.



Source page screenshot - DOCX page 29: memory architecture

How to read it: The page separates memory into specialized systems. Working memory handles immediate context; semantic memory stores facts; episodic memory stores experiences; procedural memory stores learned strategies.

Exam-important: Memory is not one thing. Exam answers should separate working, semantic, episodic, and procedural memory.

7.5 CognitiveAgent.process_input() end-to-end

Main code flow

```

process_input(user_input):
    1. Perceive input
    2. Retrieve relevant semantic and episodic memory
    3. Reason about situation and approach
    4. Create plan
    5. Execute plan with tools
    6. Adapt plan if execution fails
    7. Learn from interaction
    8. Update memory systems
    9. Generate final response

```

Exam-important: The DOCX implements the same theory from _014. If asked to explain agent architecture, use this file as the concrete implementation example.

8. _016 LangChain Multiagent

This DOCX demonstrates role-specialized agents coordinated by an orchestrator. The key pattern is not one giant agent doing everything; instead, specialized agents divide the task.

Agent/tool	Role
ResearchAgent / research_tool	Gathers information and source material.
AnalysisAgent / analysis_tool	Analyzes findings and extracts insights.
WritingAgent / writing_tool	Turns analysis into final written content.
TeamOrchestrator	Runs the workflow: research -> analysis -> writing.

LangChain multi-agent flow

Workflow:
Task/question
-> ResearchAgent gathers information
-> AnalysisAgent analyzes the research output
-> WritingAgent writes final report
-> TeamOrchestrator returns {research, analysis, report}

Exam-important: Multi-agent systems are useful when tasks naturally divide into roles. The orchestrator coordinates the specialists and passes outputs between them.

9. _017 AutoGen Single Agent

This DOCX demonstrates a single AutoGen assistant agent paired with a user proxy. The focus is conversational code generation/debugging and optional function calling.

Component	Meaning
AssistantAgent: CodeAssistant	LLM-backed assistant instructed to write Python, debug, explain, follow PEP 8, handle edge cases, and suggest best practices.
UserProxyAgent	Represents the human user; can initiate chats, manage human input mode, and execute code depending on configuration.
Termination condition	Conversation stops when configured condition is reached, such as a special message or max turns.
Function calling	Custom functions such as sentiment analysis or data fetching can be registered and called by the agent.
Example	What it teaches
Code generation	Ask the agent to write a function that removes duplicates, sorts list, and returns even numbers.
Debugging	Fix division-by-zero in average calculation when list is empty.
Optimization	Improve duplicate-finding algorithm and discuss time complexity.
Data processing	Generate random sales data, compute mean/median/std, detect outliers, display simple visualization.
Custom function calling	Agent uses registered functions instead of only generating text.

Exam-important: AutoGen emphasizes agents that converse, use tools/functions, and can execute code via a proxy agent.

10. _018 AutoGen MultiAgent

This DOCX expands AutoGen to multiple agents working together in group chat, sequential chat, coding, and two-agent collaboration patterns.

Agent	Role
User_Proxy	Represents human and can execute code.
Researcher	Gathers facts and background information.
Planner	Creates structured outlines/plans.
Writer	Creates content from research/plans.
Critic	Reviews quality and suggests improvements or approval.
Coder	Writes, runs, and debugs code with the proxy.
Pattern	Meaning
GroupChat + GroupChatManager	Agents all participate in a shared conversation managed by a chat manager.
Sequential chat	Agents work in a fixed order, e.g., Researcher -> Planner -> Writer -> Critic.
Coding task	Coder writes code; UserProxy can execute and return results/errors.
Two-agent chat	Simple handoff between specialist agents, e.g., Researcher to Writer.

Exam-important: AutoGen multi-agent workflows can be open group discussions or fixed sequential pipelines. Know the difference.

11. _019 CrewAI Single Agent

CrewAI organizes work around agents, tasks, crews, and processes. The single-agent DOCX creates a Content Creator agent with tools and several writing tasks.

CrewAI object	Meaning
Agent	A role with goal, backstory, tools, and delegation settings.
Task	A concrete job assigned to an agent with expected output.
Crew	Collection of agents and tasks.
Process	Execution style such as sequential or hierarchical.
Single-agent setup	Details
Role	Content Creator.
Goal	Create engaging, informative, SEO-optimized content.
Backstory	Expert in content creation, SEO, audience engagement, and content strategy.
Tools	WebSearch and ContentAnalyzer.
Delegation	allow_delegation=False because it is a single-agent example.
Example task	What it demonstrates
Blog post creation	Long-form content generation.
Social media campaign	Platform-specific short-form content.
Product description	Persuasive copywriting.
Email drip campaign	Sequenced marketing content.
Multi-step workflow	Research -> outline -> write -> review/optimize.

Exam-important: CrewAI frames agents as workers with role, goal, backstory, tools, tasks, and process type.

12. _020 CrewAI MultiAgent

This DOCX builds a complete marketing team. It shows how CrewAI supports sequential and hierarchical multi-agent collaboration.

Agent	Responsibility
Market Research Analyst	Market data, trends, audience insights.
Content Strategist	Content plan, channels, positioning.
Copywriter	Persuasive campaign copy.
Social Media Manager	Platform posts, engagement approach.
SEO Specialist	Keyword and search optimization.
Campaign Manager	Coordinates team in hierarchical workflow.
Tool	Purpose
MarketResearch	Returns market insights and target audience information.
SEOAnalysis	Evaluates SEO score/keywords.
SocialMetrics	Returns platform engagement-style metrics.
CompetitorAnalysis	Provides competitor positioning information.
Workflow	Steps / meaning
Product launch campaign	Market research -> content strategy -> copy creation -> social media campaign -> SEO optimization.
Hierarchical campaign	A manager agent delegates tasks and coordinates team output.
Collaborative brainstorm	Multiple agents contribute different perspectives.
Crisis response team	Assess severity -> create response strategy -> draft statement -> social response.

Exam-important: CrewAI multi-agent = role-based teamwork. Sequential process means fixed task order; hierarchical process means a manager delegates and coordinates.

13. Framework comparison

Framework	Primary idea	Best exam wording
LangChain	Connect models, prompts, tools, memory, and chains.	General LLM application framework.
LangGraph	Stateful graph workflows with cycles.	Best for agents that need loops, state, and conditional control flow.
AutoGen	Conversational agents that can collaborate, call tools, and execute code via proxies.	Best for multi-agent chat/code workflows.
CrewAI	Role/task/crew/process abstraction.	Best for role-based teams working sequentially or hierarchically.

Exam-important: If an exam asks which framework fits a cyclic agent loop, say LangGraph. If it asks role/task-based agent teams, say CrewAI. If it asks conversational multi-agent collaboration/code execution, say AutoGen. If it asks generic LLM app chains, say LangChain.

14. Key terms

Term	Short definition
Artificial Intelligence (AI)	Broad field of automating intellectual tasks normally performed by humans.
Machine Learning (ML)	AI subfield where systems learn patterns/rules from data and feedback.
Deep Learning	ML using multi-layer neural networks to learn representations.
Generative AI	AI that creates new content such as text, image, code, or datasets.
Word embedding	Numerical vector representation of words/tokens that captures relationships/context.
Tokenization	Breaking text into tokens before model processing.
LLM	Large language model trained on large text/data collections to process/generate language.
Transformer	Neural architecture using self-attention to model relationships across a sequence.
Self-attention	Mechanism letting tokens attend to other relevant tokens in context.
GPT	Generative Pre-trained Transformer; autoregressive language model family.
Autoregressive modeling	Predicting next token based on previous tokens.
Masked language modeling	Predicting hidden tokens from context, common in BERT-style training.
Prompt	Input instruction/question sent to an LLM.
Prompt engineering	Designing prompts/examples to guide output.
Fine-tuning	Additional training of a pretrained model on specific data/task.
RLHF	Reinforcement learning from human feedback; aligns model outputs with human preferences.
GAN	Generative adversarial network: generator vs discriminator.
VAE	Variational autoencoder: probabilistic encoder/latent space/decoder generative model.
MLP	Multilayer perceptron; feedforward network with input, hidden, and output layers.
Perceptron	Single neuron computing weighted sum plus bias and activation.
Weight	Learned parameter controlling input influence.
Bias	Learned offset shifting neuron activation.
Activation function	Nonlinear function applied to weighted sum.
Loss function	Measures error between prediction and target.
Optimizer	Algorithm that updates parameters to reduce loss.
Gradient descent	Optimization by moving parameters opposite gradient direction.
Backpropagation	Computes gradients through network layers to update weights.
Agent	System that perceives, maintains state, reasons/plans, acts, and pursues goals.
Cognitive loop	Perceive -> Reason -> Plan -> Act -> Learn.
Working memory	Short-term immediate context.
Semantic memory	Long-term factual knowledge.
Episodic memory	Specific past experiences.
Procedural memory	Learned strategies/skills/tool effectiveness.

15. Quick revision cheat sheet

Hierarchy:

AI -> Machine Learning -> Deep Learning -> Generative AI / LLMs

LLM pipeline:

text -> tokens -> embeddings -> transformer -> next-token probabilities -> generated output

MLP computation:

$h = f(W*x + b)$

Training loop:

forward pass -> loss -> backpropagation -> optimizer update -> repeat

Agent loop:

perceive -> reason -> plan -> act -> learn -> repeat

Compare	Remember
Classification vs generation	Classification labels an input; generation creates new content.
Pretraining vs fine-tuning	Pretraining learns general patterns; fine-tuning specializes.
Prompting vs fine-tuning	Prompting changes input; fine-tuning changes weights.
MLP vs deep network	MLP is a basic feedforward ANN; deep networks have multiple hidden layers/architectures.
Backpropagation vs gradient descent	Backprop computes gradients; gradient descent uses gradients to update parameters.
Agent vs chatbot	Agent has goals, state, tools, planning, and feedback loops.
LangChain vs LangGraph	LangChain chains apps; LangGraph adds stateful cyclic graphs.
AutoGen vs CrewAI	AutoGen emphasizes agent conversations/code; CrewAI emphasizes roles/tasks/crews/process.

16. Possible exam questions with answers

1. What is generative AI?

Answer: AI that learns patterns from data and generates new content such as text, images, code, or datasets.

2. Why are embeddings important for generative AI?

Answer: They convert words/tokens into numerical vectors, letting neural networks model meaning and context mathematically.

3. How does a GAN work?

Answer: A generator creates fake samples; a discriminator judges real vs fake. They train adversarially so the generator becomes better at realistic generation.

4. How does a VAE generate data?

Answer: It encodes input into a probabilistic latent space, samples from that space, and decodes the sample into output.

5. What does GPT stand for and how does it train?

Answer: Generative Pre-trained Transformer. It is pretrained using autoregressive next-token prediction.

6. Differentiate masked language modeling and autoregressive modeling.

Answer: Masked modeling predicts hidden tokens from context; autoregressive modeling predicts the next token from previous tokens.

7. What is prompt engineering?

Answer: Designing the input prompt, instructions, and examples to guide model output without changing model weights.

8. What is an MLP?

Answer: A feedforward neural network with an input layer, one or more hidden layers, and an output layer.

9. Write the perceptron formula.

Answer: $h = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + b$, often followed by activation $h = f(z)$.

10. Why do neural networks need activation functions?

Answer: They introduce nonlinearity; without them, multiple layers would act like one linear transformation.

11. What are loss function, optimizer, and metric?

Answer: Loss measures error; optimizer updates weights; metric reports performance such as accuracy.

12. Explain backpropagation.

Answer: It propagates error gradients backward through the network so each weight/bias can be updated to reduce loss.

13. What is gradient descent?

Answer: An optimizer method that moves parameters opposite the gradient to reduce loss.

14. What is an AI agent?

Answer: A system that perceives its environment, maintains state, reasons/plans, takes actions, and works toward goals.

15. List the cognitive loop phases.

Answer: Perceive, Reason, Plan, Act, Learn.

16. How is an agent different from traditional software?

Answer: Traditional software follows fixed rules; agents maintain context, plan, use tools, adapt, and pursue goals under uncertainty.

17. What does LangGraph add over ordinary chains?

Answer: Stateful graph workflows with cycles and conditional edges, useful for agents.

18. What is the point of multi-agent systems?

Answer: Role specialization: different agents handle research, analysis, writing, coding, review, etc., coordinated by an orchestrator or manager.

19. Compare AutoGen and CrewAI.

Answer: AutoGen focuses on conversational agents, group chats, proxies, and code execution. CrewAI focuses on agents with roles/goals/tasks/crews and sequential/hierarchical processes.

20. Name the main memory types in the cognitive-loop code.

Answer: Working, summary, semantic, episodic, and procedural memory.

17. Highest-priority focus topics

1. AI vs ML vs Deep Learning hierarchy.
2. Generative AI definition and how it differs from classification/prediction-only systems.
3. GAN vs VAE vs GPT/Transformer comparison.
4. LLM training: tokenization, embeddings, transformer, next-token prediction, pretraining, fine-tuning, RLHF, prompting.
5. LangChain basics: LLM interface, ChatOpenAI/chat messages, PromptTemplate, ChatPromptTemplate, RunnableLambda.
6. LangGraph: stateful graph, nodes, edges, conditional edges, cycles, START/END.
7. MLP mechanics: layer structure, perceptron formula, weights, bias, activation, vectorized forward pass.
8. Training loop: loss function, backpropagation, optimizer, gradient descent, metrics, train/test/deploy.
9. Agentic AI traits: autonomy, persistence, reactivity, proactiveness, adaptability, goal-orientation.
10. Cognitive loop and memory types.
11. Framework differences: LangChain, LangGraph, AutoGen, CrewAI.
12. Single-agent vs multi-agent orchestration patterns.

18. Final coverage checklist

Source file	Coverage	Status
_010_Generative AI.pptx	Covered all 27 slides: definition, deep learning base, hierarchy diagram, LSTM, GAN, VAE, GPT, ChatGPT, transformer, pretraining, prompts, tuning/domain tuning.	Yes
_011_Getting Started With LangChain.pptx	Covered all 17 slides: installation, LLM/chat interfaces, temperature/max_tokens, messages, prompt templates, chatbot runnable, LangGraph concepts and example.	Yes
_012_MLP-How it Works.pptx	Covered all 23 slides: neural network structure, tasks, learning process, perceptron math, activation, weight matrices, NumPy representation, vectorization, forward pass, gradient descent.	Yes
_013_The Context-Machine Learning - Overview.pptx	Covered all 17 slides: AI/ML/DL hierarchy, historical revival, symbolic AI, ML from data, DL representations, neuron/representation diagrams, weights/loss, parameters, deployment.	Yes
_014_The Agentic AI Revolution.pptx	Covered all 20 slides: software-to-agent shift, traits, definitions, history, architecture, separation of concerns, cognitive loop, perception/reasoning/planning/action/learning examples.	Yes
_015 Cognitive Loop LangChain DOCX	Covered 31-page code demo by module/class and main process_input flow; included memory/cognitive-loop screenshots.	Yes, summarized code not line-by-line
_016 LangChain Multiagent DOCX	Covered research, analysis, writing agents and TeamOrchestrator flow.	Yes
_017 AutoGen Single Agent DOCX	Covered assistant/user proxy, examples, function calling.	Yes
_018 AutoGen MultiAgent DOCX	Covered group chat, sequential chat, coding task, two-agent chat, roles.	Yes
_019 CrewAI Single Agent DOCX	Covered Agent/Task/Crew/Process, content creator setup, tools, tasks.	Yes
_020 CrewAI MultiAgent DOCX	Covered marketing team agents, tools, workflows, sequential/hierarchical/crisis examples.	Yes

Correction / slide note: Important diagrams were inserted inline next to the topic they explain, not dumped at the end. A few DOCX code files contain long runnable examples; these were summarized at the class/workflow level because memorizing every line is not useful for exam study. The conceptual and architectural details were covered.

Verification performed before final export: relevant files were converted/rendered to images; source slide/page images were visually checked for diagrams; guide content was organized in original source order; final PDF was rendered for layout inspection.